

Blob Compression for the Next Linea Stack

LZ4 vs. zstd vs. OpenZL — an evaluation plan

Contents

1	Blob Compression for the Next Linea Stack: LZ4 vs. zstd vs. OpenZL	2
1.1	1. The decision	2
	1.1.1 Terminology	3
1.2	2. Baseline	3
	1.2.1 2.1 Payload format	3
	1.2.2 2.2 Sizes that constrain everything	3
	1.2.3 2.3 Target stack, and what already exists	4
1.3	3. The arms	

1.8	8. Soundness	14
1.9	9. Decisions needed from stakeholders	15
1.10	10. Expected outcome	15
1.11	Appendix A — scoping measurements	16
1.11.1	OpenZL’s description language	16
1.11.2	Reproducing the key rejection	18
1.12	Appendix B — external references	18

1 Blob Compression for the Next Linea Stack: LZ4 vs. zstd vs. OpenZL

Status: research plan, not a decision. **Scope:** choose the data-availability compression scheme for the RISC-V/zkVM-verified stack. **Shape:** three sprints, each producing a usable result on its own. Sprint 1 answers the question that matters most and takes about a week.

1.1 1. The decision

Linea’s data-availability payload is compressed before being written into EIP-4844 blobs, and the rollup must prove that the blob decompresses to the payload it claims. Today that is a bespoke LZSS variant whose decompressor is hand-arithmetized as a static Plonk circuit. Moving to a Zig → RISC-V guest verified by a zkVM removes the hand-arithmetization constraint: any decoder we can write and compile is, in principle, provable. The question is which decoder we *should* write.

Better compression buys cheaper or larger blobs; more complex decoders cost proving time. The decision is that trade, and the exchange rate is set by the economics of L1 blob space and zkVM proving (§4).

Two things are already clear enough to shape the plan.

First, roughly 85–90% of the payload is transaction bodies — to be confirmed, cheaply, as the very first measurement (§5.1). Metadata is a tenth of the bytes, so however well it is compressed it cannot move the total much. **The available ratio is in the bodies.**

Second, the metadata’s redundancy is mostly the kind a match-finder already handles: repeated sender addresses, recurring block-header shapes. The exception is timestamps, whose redundancy is numeric (monotone increments) and which delta-code far better than they match — but at four bytes per block they are too small to matter. The conclusion holds on volume, not on kind.

1.1.1 Terminology

An **arm** is one candidate scheme evaluated under a common protocol: same corpus, same train/test split, same metrics. The term is borrowed from trial design because the comparison is only meaningful if every candidate is measured identically.

1.2 2. Baseline

1.2.1 2.1 Payload format

The uncompressed body is a flat concatenation of batches; only batch *byte lengths* are recorded, in the uncompressed header (`blob_spec.md`). Each batch concatenates blocks, and each block is laid out by `encode.go` as:

$$\text{block} = \underbrace{\text{numTxs}}_{\text{u16 BE}} \parallel \underbrace{\text{timestamp}}_{\text{u32 BE}} \parallel \underbrace{\text{blockHash}}_{\text{32 B}} \parallel \underbrace{(\text{from} \parallel \text{txRLP})^{\text{numTxs}}}_{\text{20 B}}$$

Two levels of nesting are *implicit*: block counts are recovered by scanning until a batch’s byte budget is exhausted (`ScanBlockByteLen`), and transaction lengths by decoding RLP list headers (`passRlpList`). This matters only in §7, where it bounds what `OpenZL` can express.

1.2.2 2.2 Sizes that constrain everything

From `blob_maker.go`, with `PackingSizeU256 = 254`:

- usable payload per blob after field packing: $B = 4096 \times 254/8 = 130\,048$ bytes;

- `MaxUncompressedBytes = 780 000` — “the max we can do ... with 2^{27} constraints”.

So the deployed circuit is provisioned for a ratio of $780\,000/130\,048 \approx 6.0$ and **cannot benefit from anything better without being resized**. The ratio ρ_0 actually achieved on mainnet is an input to measure, not assume.

1.2.3 2.3 Target stack, and what already exists

The pipeline is `Zig` $\rightarrow$ `rv64im ELF` $\rightarrow$ `elf_to_json` $\rightarrow$ `zkc exec` $\rightarrow$ cycle count $\rightarrow$ AIR trace. Three pieces of infrastructure are already in place and should be reused rather than rebuilt:

- **Guest layout.** `riscv-guests/` hosts self-contained Zig guest packages; its README anticipates further guests slotting in the same way.
- **Cycle measurement.** `bench_compress` builds a guest, runs `zkc exec --fast`, subtracts an empty-loop baseline and emits CSV; `verifier-profiling.md` documents the VERIFIER-MARK protocol. **This is the template for all cycle work below.**
- **Accelerators.** Poseidon2 and Keccak exist as custom RISC-V opcodes (`lineth-accelerators`), so payload hashing does not run in software.

1.3 3. The arms

Arm	Scheme	Decoder to build and prove	Sprint
A0	Current LZSS + trained dictionary	LZSS	1
A1	LZ4 + dictionary	LZ4	1
A2	DEFLATE + dictionary	LZ77 + canonical Huffman	1
A3	zstd + trained dictionary, restricted feature set	restricted zstd	1

Arm	Scheme	Decoder to build and prove	Sprint
A4	Re-encoded payload + zstd	A3's decoder + inverse permutation (+ RLP re-encoder)	3
A5	OpenZL	<i>none — measurement only</i>	3

1.3.1 3.1 Why DEFLATE is in the set

DEFLATE is not merely a fallback if zstd proves too expensive, though it is that too. Its real value is that it turns a two-point comparison into a three-point curve and separates two costs that would otherwise be confounded:

- **A1** → **A2** isolates the cost and benefit of *entropy coding as such*: LZ77 plus canonical Huffman, 32 KB window, a decoder of order 500–1000 lines.
- **A2** → **A3** isolates zstd's specific machinery: FSE/tANS alongside Huffman, three separately tabled sequence streams, repeat-offset history, larger windows, several block types.

If the ratio gain is mostly in the first step and the cycle cost mostly in the second, DEFLATE is the answer and we will be able to say so with evidence rather than intuition. Note that zlib supports trained dictionaries, so A2 can be compared on equal footing with A0 and A3.

For completeness on the family: LZMA/7-Zip sits *above* zstd in zkVM cost despite being older, because its range decoder is a serial arithmetic-coding loop with per-bit adaptive probability updates — long dependent-arithmetic chains per output bit. It is not proposed as an arm. zstd is a good stand-in for the modern LZ-plus-entropy family; it is not a stand-in for the context-modelling family.

1.3.2 3.2 Why OpenZL is measured rather than ported

OpenZL's value proposition is structural decomposition of the bodies, which requires either a re-encoded payload or a custom parser. Its honest competitor is therefore A4, not A3 — and A4 captures the same structural win with a decoder

we fully control. OpenZL's *marginal* value over A4 is its specialised numeric codecs applied to those same decomposed fields: real, but narrower than the general pitch, and paid for with a graph interpreter plus roughly 55 transforms inside consensus-critical code.

So A5 is run **host-side only**: no Zig port, no zkVM measurement. What it buys is a reference point. If we hand-roll a decomposition and get some ratio, we have no way of knowing whether that is near the ceiling or half of it; OpenZL trained on the same corpus tells us how much exploitable structure is actually present. If A4 lands close, stop. If A4 lands far below, the gap is the justification for paying for a custom parser, and §7.3 reopens with a number to defend it.

The floor argument — that OpenZL is no worse than zstd, since it can select the zstd leaf codec — is true and not load-bearing. We would likely forbid that leaf anyway to keep the verifier small (§5.3), and a floor is only interesting if the ceiling is.

1.4 4. What a ratio improvement is worth

1.4.1 4.1 The break-even inequality

Write $B = 130\,048$ for usable bytes per blob, ρ for compression ratio, κ for zkVM cycles per uncompressed byte of decoding, π for proving cost per cycle in USD, P_{L1} for the L1 cost attributable to one blob, and Γ_0 for per-blob fixed cycles. Cost per uncompressed payload byte is

$$c(\rho, \kappa) = \frac{P_{L1} + \pi \Gamma_0}{\rho B} + \pi (\kappa + \kappa_{\text{fix}}),$$

where κ_{fix} covers hashing, packing and blob consistency and is the same for every arm. Arm X beats the baseline exactly when

$$\boxed{\frac{P_{L1} + \pi \Gamma_0}{B} \left(\frac{1}{\rho_0} - \frac{1}{\rho_X} \right) > \pi (\kappa_X - \kappa_0)}$$

so a ratio improvement purchases a cycle budget of

$$\Delta\kappa^* = \frac{P_{L1} + \pi\Gamma_0}{\pi B} \left(\frac{1}{\rho_0} - \frac{1}{\rho_X} \right) \text{ cycles per uncompressed byte.}$$

Note that ratios themselves are misleading here; the **reciprocal difference** is what pays, and that is what every table should report.

1.4.2 4.2 Sensitivity, and why the blob fee dominates

Blob gas per blob is $2^{17} = 131\,072$, so at blob base fee f the blob costs $131\,072f$. Taking $\rho_0 = 5$, $\rho_X = 6$, a GPU prover at roughly 10^7 cycles/second costing \$2/hour (hence $\pi \approx 5.6 \times 10^{-8}$ USD/cycle), and \$4000/ETH — all placeholders to be replaced by sprint-2 measurements:

Blob base fee	P_{L1} per blob	Affordable $\Delta\kappa^*$
1 wei (protocol floor)	$\approx 5 \times 10^{-10}$ USD	essentially zero
1 gwei	$\approx \$0.52$	≈ 2 cycles/byte
30 gwei	$\approx \$16$	≈ 70 cycles/byte
300 gwei	$\approx \$157$	≈ 700 cycles/byte

The decision is dominated by the empirical distribution of the blob base fee, not by any property of the algorithms. When blob space is uncongested the fee sits at its floor, DA is nearly free, and no increase in decoder complexity is justified by any ratio improvement. What matters is the frequency and depth of congestion episodes, not the median.

1.4.3 4.3 Capacity may matter more than cost

There is a second channel, and it behaves differently. Blobs per L1 block are a hard protocol resource, so Linea’s maximum sustainable throughput is $\rho \cdot B \cdot$ (blobs per second Linea can claim). Here the ratio multiplies the throughput ceiling **regardless of the current blob price**.

If the binding constraint is throughput rather than cost, §4.2’s table is the wrong table: a ratio improvement is then worth whatever the marginal L2 transaction is worth. Sprint 2 must produce both valuations and say which governs. This is

a product question as much as an engineering one and should be put to stakeholders in week one (§9).

1.4.4 4.4 Feasibility gates, independent of cost

- **Segment budget.** Total cycles must fit the segmentation strategy at acceptable latency. A decoder that is affordable but puts the DA proof on the critical path of finalisation is still unacceptable.
 - **Memory.** Window, entropy tables and output buffer must fit; guest memory is arithmetized and scarce. zstd's window is a tunable parameter here, not a constant.
-

1.5 5. Sprint 1 — is this practical, and roughly worth it?

Goal. Decide whether a zstd-class decompressor is viable in the zkVM at all, and whether the ratio gain is in a range that could ever pay. Roughly a week. **This sprint answers the highest- value questions and its output stands alone,** whether or not sprints 2 and 3 happen.

Crucially it needs **no corpus tooling and no encoding work**. The tree already contains `testdata/v0/sample-blob-*.bin` and 27 prover-response fixtures under `testdata/v1/`, which `DecompressBlob` turns back into real payloads. That is enough for a first ratio number on day one.

1.5.1 5.1 Payload anatomy (half a day, do it first)

For the available payloads, report the byte split across block metadata, sender addresses and transaction bodies; within bodies, the split across RLP fields with calldata separated; the share consumed by `blockHash`, which is 32 incompressible bytes per block and a floor no scheme can touch; and transactions per blob.

If bodies are not the overwhelming majority, §1's framing is wrong and §3 needs rework. Better to discover that in half a day than in week three.

1.5.2 5.2 Crude ratios (1 day)

A0 against A1, A2 and A3 on those payloads, each with a trained dictionary so the comparison is honest — comparing an undictionaryed zstd against a dictionaryed LZSS is the standard way to make this exercise meaningless. Report medians with inter-quartile range, since blob economics are driven by the distribution: one payload that compresses badly forces an extra blob.

Sweep zstd’s window size and record the ratio-versus-window curve rather than a point, because window size is a *decoder memory* parameter and feeds §4.4.

Stated limitation. These arms are measured on the *current* payload encoding. That is roughly exact for A0 and A1, which have no entropy layer to starve, and a **lower bound** for A2 and A3. See §7.1 — the deferral is deliberate but it does bias against the more sophisticated codecs, and sprint 1’s numbers must not be read as closing the question.

1.5.3 5.3 Pin the zstd feature set (half a day, a required deliverable)

We control the compressor, so we choose which features the decoder must support; anything else it *rejects*, which is sound provided rejection is a proof failure and never a silent acceptance. The decoder cannot be written until this is pinned. Candidate restrictions:

- single frame, single segment, known content size, no checksum;
- fixed window size, chosen from §5.2’s curve;
- Huffman-only literals with a pinned table, or raw literals, avoiding the FSE literal path;
- a closed set of block types, with everything else rejected.

Each restriction removes decoder code, cycles and audit surface. **Measure the restricted decoder, not libzstd.** scroll-tech/rust-zstd-decompressor is a light-weight decoder written for exactly this purpose and worth reading first.

1.5.4 5.4 Cycle costs (3–4 days)

Create a data-availability guest per the guests README and copy the harness from bench_compress. Measure, in order:

1. κ_{fx} **frst** — a guest that only unpacks the blob, hashes the payload and

checks blob consistency. One to two days, and it may end the investigation early: if $\kappa_{\text{fix}} \gg \kappa_{\text{dec}}$ for every arm, then no candidate decoder is “overwhelming”, question (1) answers itself, and the decision collapses to maximising ρ subject to §4.4. Poseidon2 has a custom opcode, so this should be low — but the *arithmetized* cost of that opcode is not the same as its cycle cost, and both need looking at.

2. **A1** — a minimal LZ4 block decoder in Zig. Small; establishes the floor.
3. **A0** — the LZSS decoder, for an apples-to-apples baseline in the new stack. Without it the comparison is against a Plonk-circuit cost that no longer applies.
4. **A2**, then **A3** — the two-step curve of §3.1. A3 is the long pole.

Report cycles broken down by phase, not as one number: bit reading, entropy-table construction, symbol decode, match copy, literal copy. This is what makes §5.5 decidable on evidence rather than intuition, and it costs nothing extra given the marker-based harness.

1.5.5 5.5 Custom opcodes: where the headroom is

Decompression is not uniformly “basic CPU work”; three inner loops are plausible accelerator candidates, in rough value order:

- **Match copy** — copy L bytes from distance D , byte-at-a-time whenever $D < L$ because the copy overlaps. This is the LZ inner loop and it benefits *every* arm including LZ4.
- **Bit reader** — zstd and DEFLATE both read variable-width bitfields; on RISC-V that is several shift/mask/load instructions per symbol.
- **Entropy symbol decode** — table lookup plus state advance, fusable into a single operation.

The second and third are exactly what makes zstd expensive relative to LZ4, so **accelerating them would substantially compress the difference between the arms** and could change the ranking. Set against that, every opcode is arithmetization work and trace columns in zkC: the usual precompile economics, worth it only if the operation dominates. §5.4’s phase breakdown is what tells us whether it does.

1.5.6 5.6 What sprint 1 does and does not establish

Cycle counts rank candidates and catch order-of-magnitude problems, which is all that is needed to kill arms. They are **not** sufficient for final costing: zkC lowers to AIR, and cost per cycle varies with instruction mix, so memory-heavy and arithmetic-heavy decoders do not cost the same. One real trace-length and proving-time measurement is required for the surviving arm — deferred to the end of sprint 3.

Known dependency: `bench_compress` records that tracing mode currently panics under `KOALABEAR_16`, the 32-bit instruction register exceeding the 16-bit field register width. The `--fast` path used for cycle counts is unaffected, but that panic must be fixed before any full proving measurement is possible. **Raise it early; it is on someone else's critical path too.**

1.5.7 5.7 Sprint 1 deliverable

One page: payload anatomy, ratio table with $\frac{1}{\rho_0} - \frac{1}{\rho_x}$ per arm, cycle cost per arm with phase breakdown, the pinned `zstd` feature set, and a recommendation on custom opcodes. Enough to decide whether to continue.

1.6 6. Sprint 2 — what it is worth in money

Goal. Populate §4 with measured inputs. About three days, and it can run in parallel with the tail of sprint 1.

1. **Corpus.** Sprint 1's fixtures are adequate for triage, not for a decision. Build a reproducible corpus spanning quiet and congested periods, `calldata-heavy` and `transfer-heavy` traffic, and several blob-fee regimes. Check whether `consensus/blobsim` — private, Go, with `block_producer.go`, `blockdb.go` and `connect_linea_test.go` — is a foundation before standing up a node; the fallback is fetching blocks over JSON-RPC and feeding `blob.NewBlobMaker` directly, which is a short program. **Train dictionaries on a disjoint prefix and evaluate on a held-out suffix;** the train/test gap is largest for exactly the arms that train hardest.
2. **Blob fee distribution.** Historical blob base fee over a horizon matching the decision's intended lifetime, reported as a distribution rather than a mean,

with known protocol changes to blob targets and limits stated as explicit assumptions.

3. **Linea’s blob consumption.** Blobs per day, payload bytes per blob, fraction of capacity used, from the coordinator’s submission history.
4. π . Measured prover throughput in cycles/second on target hardware, times that hardware’s cost per second; cross-checked against current end-to-end cost per proof.
5. **Regime determination (§4.3).** Is throughput blob-limited today, and if not, at what traffic level would it become so? This decides which valuation governs.

Deliverable. $\Delta\kappa^*$ under each fee scenario, and therefore which arms from sprint 1 are affordable. In many outcomes this is where the project ends with a decision.

1.7 7. Sprint 3 — encoding and OpenZL, conditional

Runs only if sprints 1 and 2 show ratio headroom worth chasing. One to two weeks.

1.7.1 7.1 The non-orthogonality caveat

Payload re-encoding is *not* orthogonal to the choice of compressor, and the interaction runs in one direction: **transposition’s value scales with the sophistication of the entropy and numeric layer.** It does close to nothing for LZ4, which has no entropy coder to feed; it helps DEFLATE and zstd, whose entropy statistics improve when like fields are adjacent; and it is the entire point for OpenZL, whose numeric codecs only function on homogeneous typed streams.

The consequence for sequencing: deferring this work biases sprint 1 *against* the more sophisticated codecs. That is the right trade under time pressure, but it means an A3 that looks unattractive in sprint 1 might look different here, and that possibility should not be forgotten.

1.7.2 7.2 Arm A4, at two depths

A4a — metadata transposition. Counts, timestamps, hashes and sender addresses to columns; bodies untouched. Nearly free to implement and, by §1,

expected to be worth little. It is a *control*: a large gain here means §1 is wrong.

A4b — field-level decomposition. Transactions decoded into width-normalised parallel columns — nonce, gas limit, fee fields, recipient, value — with calldata as a lengths array plus one concatenated region. This is where delta coding, tokenization and bitpacking become applicable at all: nonces and timestamps delta to near nothing, recipients tokenize well because a handful of contracts dominate L2 traffic, and value fields are mostly zero-padded u256.

A4b’s cost is not the transposition; it is that the payload no longer stores RLP, so the decoder must **re-encode to byte-exact RLP** to preserve hashes (§8). Weigh against it the risk that the deployed LZSS-plus-dictionary is already producing long matches spanning whole near-identical transactions, which column-splitting would fragment. Whether column-wise entropy coding more than compensates is genuinely uncertain, and measuring it is the point.

A practical note for either depth: a transposed encoding removes `blob_maker.go`’s incremental blob fill, since appending a block perturbs every column. This is a solved problem — the fit predicate is monotone in block count, so binary search costs about eleven full compressions per blob instead of one per block — and the existing code already contains a whole-payload recompression path. Confirm the latency cost; do not let it steer the encoding choice.

1.7.3 7.3 Arm A5, the OpenZL oracle

Run `zli train` plus a generic profile for the baseline number, and — if the probe below succeeds — a description over the A4b encoding. **Report the A4b-to-OpenZL gap explicitly**; that single number decides whether §3.2’s reclassification stands.

Write the readmission threshold down *before* the numbers arrive, using §4.1: the gap must imply a $\Delta\kappa^*$ large enough to pay for a graph interpreter plus the exercised transform set, under a fee scenario sprint 2 considers plausible.

One probe gates the cost of this arm. Can a lengths stream drive the splitting of a concatenated bodies region in the codec graph — via `splitN`, `dispatch_string` or the variable-size-field machinery — and at which layer must that be expressed? Half a day. If it works, A5 is a description plus configuration over A4b’s encoding and is cheap. If not, A5 needs a hand-written

custom C parser (`custom_parsers/` already contains Parquet, PyTorch, CSV and ZIP lexers, so the extension point is real): 3–4 days, and worth writing only once A4b and the generic OpenZL numbers are in hand.

If and only if OpenZL is readmitted as a decoder candidate, the following become live: frame-format stability as documented policy rather than incidental behaviour, which a rollup needs because a DA decoder is effectively immutable once deployed; decode-side memory against the documented $\sim 10\times$ payload figure; the size of a Zig port of the exercised transform subset plus the graph interpreter; and whether the codec set can be restricted to a whitelist the decoder soundly enforces, as in §5.3.

Appendix A records what scoping already established about OpenZL’s description language, including the one pattern it cannot express.

1.8 8. Soundness

Proving decompression is half the obligation; the decoder’s failure modes must all be proof failures, never silent acceptances. Three hazards, all worsening as decoders get more expressive:

- **Byte-exact RLP re-encoding.** A4b stores decoded fields, so the decoder must reproduce the original RLP byte for byte — minimal-length integers, correct list-header widths, type-envelope bytes. Any divergence changes transaction hashes. **This is the largest new soundness surface the plan introduces, and it is introduced by the arm most likely to win on ratio.** Differential fuzzing against a reference encoder is mandatory.
- **Rejection completeness.** Every restriction pinned in §5.3 must be enforced *by the decoder*, with violations aborting the proof. A decoder that ignores an unsupported flag rather than rejecting it is a soundness bug.
- **Malleability.** If two distinct frames decode to the same payload, or one frame decodes differently under different decoder versions, the DA commitment is not binding in the way the rest of the protocol assumes. `zstd` and OpenZL both have far more representational freedom than LZSS.

Budget review and differential-fuzzing time for the winning arm. This cost scales with decoder complexity and belongs in the comparison, not outside it.

1.9 9. Decisions needed from stakeholders

Ask in week one; each changes the shape of the work.

1. **Capacity or cost?** Is near-term throughput expected to be blob-limited (§4.3)? If yes, §4.2's table does not govern and the bar for complexity drops sharply.
 2. **Horizon.** Over what period must this decision hold? One year and five years give different answers on OpenZL's maturity risk.
 3. **Payload format.** Is changing the uncompressed encoding acceptable, given it touches the coordinator, blob maker and state recovery? A "no" removes A4 and most of A5, and reduces the plan to sprints 1 and 2.
 4. **Upgradeability.** Can the DA decoder be versioned and replaced post-deployment? If yes, the cost of choosing conservatively now falls sharply and shipping A1 or A2 first becomes attractive.
 5. **Latency budget.** What wall-clock contribution of the DA proof to finalisation is acceptable? This bounds κ independently of cost.
-

1.10 10. Expected outcome

Stated in advance so we notice if we talk ourselves into it. The most likely result is that a restricted zstd (A3) is affordable and wins, with DEFLATE (A2) as the fall-back if zstd's entropy machinery proves disproportionately expensive in cycles, and with A4b held in reserve as the ratio play if sprint 2 shows the economics justify it.

Four ways that could be wrong, each of which the sprint structure is designed to catch early:

- **§5.1 shows bodies are not dominant** — the framing in §1 collapses and §3 needs rework.
- **κ_{fx} dominates** — decoder cost is irrelevant, and the decision becomes "maximise ρ ", pushing toward A4b and possibly OpenZL.
- **The blob-fee distribution is benign** — A1 is correct and the exercise argues for *less* complexity than we have today.

- **Custom opcodes fatten the curve** (§5.5) — the ranking between A1, A2 and A3 changes once the bit reader and symbol decode are accelerated.

1.11 Appendix A — scoping measurements

Recorded so the work is not repeated. From the scoping pass preceding this plan.

Fact	Value	Source
Usable payload bytes per blob	$4096 \times 254 / 8 = 130\,048$	<code>PackingSizeU256 = fr381.Bits - 1</code>
Current uncompressed cap	780 000 B at 2^{27} constraints	<code>blob_maker.go:27</code>
Implied maximum useful ratio	≈ 6.0	derived
Poseidon2 software compress	8 657.6 cycles/call	<code>bench-compress.csv</code> ; <i>software path — a custom opcode exists and should be used instead</i>
OpenZL standard decoders	~ 55	<code>decoder_registry.c</code>
OpenZL version tested	dev @ adadd5c, zli 0.2.4	local build
OpenZL library memory	$\sim 10\times$ payload, no streaming, 500 MB cap	<code>library-limitations.md</code>

1.11.1 OpenZL’s description language

The published SDDL documentation is **aspirational** and describes intended rather than implemented behaviour; a separately installed zli predating the current syntax failed even on descriptions shipped in the same tree. Every probe below was run against the commit named above, and any future probe must record its own.

Pattern	Result
Nested records; arrays with data-dependent counts; sizes read at top level and passed in as parameters	work
Fully columnar payload: counts, then parallel fixed-stride arrays, then one bodies region	works, round-trips
Union by record <i>parameter</i> ; pre-dispatch into one fixed-layout array per type	work
Record reading its own length (record Tx() { len: UInt16BE, body: Bytes(len) })	rejected: Scan records are not yet supported
Size read in the <i>enclosing</i> record and passed down	rejected: same error
Array indexing (Bytes(lens[0]))	rejected
Union by <i>per-element</i> discriminator read from data	rejected: scan record
Top-level manual unroll of k variable-length items	works for $k \leq 254$; VM has 256 registers
Decompressing a frame <i>without</i> supplying the description	works

The conjecture these support. The top level is the only scan context. Within it, data-dependent sizes and counts work to arbitrary nesting depth provided the dependence is resolved before entering a record and passed in as a parameter. Inside a record, no field may influence that record’s own layout. **Repetition of self-describing elements is therefore inexpressible** — it needs a variable-stride array or a backward branch, and the VM (`Assembly_opcodes.md`) has neither: `jump_if` only skips forward, the `CALL` category is empty, and `type.fixed_array` multiplies a single element width by a count.

Nesting is not the obstacle; **variable-length slices under repetition** is. For the deployed payload that is exactly the transaction stream (§2.1).

Two consequences. First, the columnar encoding that sidesteps this is the

same one arm A4 adopts for ratio reasons — the encoding that makes the format expressible is the encoding that gives entropy coders something to work with. Second, since a frame decompresses with no description supplied, the description bounds *which decompositions are reachable* and hence the achievable ratio, but never appears in a decoder’s cost. That is why OpenZL belongs in a ratio measurement and not in cycle work.

1.11.2 Reproducing the key rejection

```
cat > scan.sddl <<'EOF'
record Tx() {
    len: UInt16BE,
    body: Bytes(len)
}
tx: Tx
EOF
zli compress -p sddl2 --profile-arg scan.sddl input.bin -o /dev/null -f
```

1.12 Appendix B — external references

- OpenZL — <https://github.com/facebook/openzl>, documentation at <https://openzl.org/>
- Scroll’s light-weight zstd decompressor for DA blobs — <https://github.com/scroll-tech/rust-zstd-decompressor>
- consensys/compress (current LZSS) — <https://github.com/consensys/compress>
- gnark’s in-circuit LZSS decompressor — <https://github.com/consensys/gnark/tree/master/std/compress/lzss>
- consensys/blobsim — private; evaluate in sprint 2